



Security Controls in Shared Source Code Repositories

PRESENTED BY JUSTIN MORROW

CSD-380: MODULE 10.2

DATE: 07/25/2026



ZERO TRUST

Controlling the repository entry points prevents unauthorized access and maintains strict compliance:

- **Role-Base Access Control (RBAC):** Enforces the principle of least privilege, restricting admin permissions to designated personnel.
- **Mandatory MFA & SSO:** Requires multi-factor authentication and enterprise IdP integration to eliminate single-point credential compromises.
- **Auditability & Governance:** Centralized logs track exact user commits and policy modifications for regulatory compliance.

DevOps Security Practices



Defensive Architecture:

Best when repository pipelines have a combination of RBAC, MFA, Branch protection, secret scanning, & automated SAST/SCA testing.



Security Development:

Shifting security left ensures continuous compliance by embedding security checks in the earliest stages of the Software Development Lifecycle (SDLC).

Repository Security Control Pipeline



Identity Validation

Developer cryptographic verification for signed commits (GPG/SSH)



Validation Checks

Run automated CI security scans



Branch Protection

Enforce approvals before code can merge with the production branch

Feature Branch

Pull Request

Branch Protection

Cryptographic Commit Signing

Automated Gates

Merge to Main Branch

Automated Security Testing

SAST Analysis

Static Application Security Testing scans the source code during commits to detect vulnerability patterns early in the SDLC.

SCA Scanning

Software Composition Analysis inspects 3rd party open-source libraries for known vulnerabilities and license compliance risks.

Shift Left

Embedding automated scans into pre-commit and CI gates provides instant developer feedback and reduces remediation costs.



Governance to Ensure Code Integrity

- Protecting the main branch ensures that no unvetted or malicious code will enter the production workflow.
- Enforcing mandatory pull requests and approval rules will ensure oversight prior to merging branches.

Preventing Secret Exposures

- **Eliminate Hardcoded Credentials:** Ensuring Developers don't inadvertently commit API tokens, DB passwords, SSH keys, or certificates into git history.
- **Pre-Commit Hooks:** Intercept local commits containing credential patterns before pushing to remote servers.
- **Automated Repository Scans:** Continuously monitor commit histories to immediately flag and revoke leaked keys.
- **Secret Vaults:** Centralized key management tools.

Online Resources

- CHECKMARX. (JUNE 11, 2025)
 - [Repository Health Monitoring Part 2: Essential Practices for Secure Repositories](#)
- HARNESS. (FEBRUARY 13, 2025)
 - [What Is Code Repository Security?](#)
- SNYK. (N.D.)
 - [Securing Source Code in Repositories is Essential: How To Get Started](#)
- IBM. (N.D.).
 - [Shift-left security](#)